SNHU Travel Sprint Review and Retrospective

Youssef Amor

Southern New Hampshire University

CS 250 Software Development Lifecycle

Anthony Kutscher

October 18, 2026

SNHU Travel Sprint Review and Retrospective

The SNHU Travel simulation showed me why Scrum fits a product whose requirements develop through feedback. My strongest evidence is the progression from user needs to test cases and then revised slideshow tests. As Scrum Master, I would use the Sprint Review to inspect product progress and the Retrospective to improve how we work. Because the course simulated a team, this reflection distinguishes completed coursework from proposed team practices.

Contributions of the Scrum Roles

The Product Owner contributed direction by connecting features to customer value. In my Module Three journal, I explained how user stories capture both a request and its purpose . The focus group made that concrete: customers requested ranked destinations, personalized recommendations, and price limits. Christy could use that feedback to order the Product Backlog rather than treat every request as equally urgent.

My developer reflection emphasized asking for priorities and acceptance criteria before changing implementation . My tester work translated expectations into observable checks, including destination counts and image-description consistency. Testing belongs within the Developers accountability in Scrum, although the course separated these activities for practice. In Module Two, I proposed a Scrum Master approach that protects realistic goals, removes impediments, and encourages participation . Together, these contributions connect customer intent with implementation and quality rather than leaving each role to make separate assumptions.

Completing User Stories

Scrum helped me connect requirements, design, and testing in a repeating development cycle. The focus group supplied the customer problem, while user stories explained the desired benefit. For example my workbook linked to the top-five destination story, to package details, and the mobile application then converted the budget concern into checks that a price-range filter excludes packages outside the selected range and that opening a package displays the correct details.

Then when we went in and revised we checked a different interaction: one destination at a time, ordered navigation, and matching images and descriptions. The Revision carried those expectations into mobile navigation. This traceability helps a story reach completion because the developer and tester can share a concrete target.

Handling Interruptions and Changes

The documented change was the move from a destination list to a slideshow. My tester journal explains that seeing the updated slideshow helped me revise the tests. I replaced list-oriented checks with Next and Previous actions while keeping the underlying need to browse destinations. Scrum supports this adjustment through backlog refinement and discussion with the Product Owner. During a Sprint, Developers can negotiate scope with the Product Owner while protecting the Sprint Goal. If the change threatens that goal, I would make the tradeoff visible rather than quietly add work. The revised tests demonstrate adaptation, although the materials do not establish a particular delay or recovery time.

Communication and Collaboration

My coursework includes communication samples that address uncertainty before it becomes rework. In the tester email to Christy, I asked, “For the Top Destinations feature, should the destinations appear on one page or as a slideshow?” I also asked how many destinations to include, their order, and whether both navigation buttons were required . These questions were effective for the situation because each answer would change a specific test. Addressing Christy directly connected the uncertainty with the Product Owner who could clarify the intended experience.

My developer email used the subject “Clarification Needed for Updated SNHU Travel Requirements.” I asked the Product Owner to confirm priorities and acceptance criteria and asked the tester to identify expected results and edge cases. This sample encouraged collaboration by giving each recipient a clear contribution. It also connected requirements to development tasks, making it easier to identify obsolete work before implementing changes.

These are emails written for the simulation, rather than evidence of messages sent to an actual team. Their strength is the specific request for a shared understanding. I would improve the developer sample by naming the affected story IDs, requesting a response before the next planning decision, and recording the agreed answers beside the backlog items. I would also ask about behavior at the first and fifth slides, which the revised tests do not explicitly resolve. Summarizing the answer back to Christy and the tester would confirm that we interpreted it consistently. A brief follow-up discussion could settle disagreements, with the final decision preserved in writing.

Organizational Tools and Scrum Events

The Excel workbook linked story IDs and priorities to preconditions and expected results. Its original and revised tabs exposed the interface change. However, duplicate tabs can become outdated, and the workbook lacks execution outcomes. I would designate current tests and record actual results. An ordered Product Backlog would support Sprint Planning, while a shared task board would help Developers inspect progress during the Daily Scrum. These are proposed tools. Refinement would clarify upcoming stories. At the Review, working software and test evidence would support stakeholder discussion; at the Retrospective, the revision history would expose preventable mistakes

Evaluation and Next Steps

Scrum suited SNHU Travel because customer expectations changed. Short feedback cycles let the team inspect whether a slideshow or price filter serves travelers before expanding the product. Transparency and adaptation supported test revision. The costs were repeated clarification and testing. Weak acceptance criteria could increase rework, while accepting every request could undermine focus. Events would also consume time without value if they produced no decisions.

I consider Scrum the best fit because the experience evolved and could be evaluated incrementally. A sequential approach would offer clearer upfront baselines, but changing an approved interface would require revisiting design and test plans. For the next Sprint, I would have the Product Owner and Developers clarify slideshow boundaries before coding, record test outcomes, and revisit these improvements at the Retrospective. ChadaTech should validate the approach in a real pilot before generalizing from this individual simulation.